

CGI Capacity Analyzer for Director Workload and RFP Assignment

Freya Kan, Jai, Junxian Lin, Qian Yang, Yixiao Jing
Master of Data Science in Computational Linguistics
University of British Columbia

Abstract

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. We develop a Capacity Analyzer that combines historical CRM opportunity data with proposal and RFP text to support director capacity estimation and RFP assignment. The system is designed as a transparent decision-support tool rather than a replacement for business judgment. Week 1 established the reproducible opportunity data foundation and module skeletons. Week 2 extended this foundation into a first integrated prototype by refining the cleaned opportunity dataset, validating scoring fields, implementing director-level capacity scoring, connecting capacity outputs to the Streamlit dashboard, and advancing the RFP preprocessing pipeline. Week 3 refined the capacity dashboard around a relative-load-first interpretation and shifted the main focus toward RFP assignment, including fallback retrieval, assignment schema stabilization, heuristic recommendation logic, and Streamlit RFP page integration. Week 4 expanded the prototype into a final stakeholder-facing version with real capacity-aware RFP recommendations, Azure OpenAI chat enrichment for RFP summaries, effort estimates, and director match reasons, Azure OpenAI embedding and Chroma retrieval integration with local fallback preserved, RFP upload support alongside paste input, clearer fallback and prototype-status labeling, and stronger empty-state, error-state, and response-quality checks. The resulting prototype remains a transparent decision-support system rather than a production-ready or business-validated assignment system.

1 Introduction

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. Today, capacity may be tracked manually or inconsistently, making

it difficult to see which directors are available, near their historical workload norm, overloaded, or well matched to a new opportunity.

This project builds a CGI Capacity Analyzer to support two related business decisions. First, the Director Capacity Dashboard summarizes each director's current workload, historical baseline, relative load, capacity label, and workload drivers. In the Week 4 prototype, `relative_load` remains the primary business-facing metric because it compares a director's current workload against that same director's historical norm. Second, the RFP Assignment Tool estimates RFP effort, retrieves similar historical proposal examples through the Azure OpenAI embedding and Chroma retrieval path when configured, and recommends suitable directors based on available capacity signals, retrieval evidence, service-domain overlap, and risk. The local fallback retrieval path remains available when Azure or Chroma is unavailable.

The system is not intended to replace stakeholder judgment. Instead, it provides a transparent and reproducible data-based starting point that CGI stakeholders can review, validate, and adjust. This design choice is important because the project does not currently have manually labeled ground truth for director capacity, RFP effort, or correct director assignment. The Week 4 RFP outputs should therefore be interpreted as prototype guidance: capacity-aware when real capacity data is available, LLM-enriched when Azure chat configuration is active, and still supported by deterministic heuristic or fallback behavior when LLM enrichment or retrieval components are unavailable.

2 Data

2.1 Data Sources

The project uses two anonymized Excel CRM opportunity exports and one JSON file containing historical proposal and response text. The two oppor-

tunity exports are referred to as opps1 and opps2. The proposal data contains historical proposal and response document pairs.

All CGI-provided files are treated as sensitive local data. They are not committed to GitHub. Local credentials and API configuration are also excluded from version control.

2.2 Opportunity Data

Initial EDA indicates that opps1 and opps2 are different views of largely overlapping CRM opportunity data. We use opps2 as the primary base table because it contains the richer CRM schema and has one unique opportunity_id per row in the current export. We use opps1 as a secondary source: selected fields from opps1 are collapsed to one row per opportunity_id and merged into opps2, while opps1-exclusive opportunities are appended to the final opportunity_df. Fields that appear under different names across the two exports, especially revenue-related fields, require normalization and validation before downstream scoring and are handled through the Week 2 validation and fallback process.

This distinction is important because the downstream capacity model is expected to operate at the opportunity level. During the Week 1 merge implementation, we found that opps1 contains duplicate opportunity_id values. These duplicates likely reflect service-level or product-level detail rather than distinct opportunities. A direct merge from opps1 into opps2 would therefore multiply some opportunity rows and risk double-counting workload in later capacity scoring.

To preserve the intended data grain, the merge pipeline collapses opps1 supplemental fields to one row per opportunity_id before joining them to opps2. Duplicate and detail rows from opps1 are preserved in local audit outputs rather than being used to expand the final opportunity_df.

2.3 Proposal and RFP Text Data

The proposal JSON contains 19 historical proposal and response document pairs. These documents are text-heavy and many are too long to embed as single inputs. Therefore, the RFP pipeline prepares proposal text through text extraction, chunking, metadata construction, and retrieval-oriented processing. The historical or sample corpus is chunked before retrieval so that retrieved evidence can be surfaced as smaller supporting examples rather than as full proposal documents.

By Week 4, the RFP pipeline loads and chunks the local proposals_responses.json corpus when available, with a built-in sample corpus used only as fallback. Submitted RFP text, whether pasted or extracted from TXT, DOCX, or text-based PDF upload, is also chunked before retrieval. The preferred retrieval path uses Azure OpenAI embeddings and Chroma when configured and suitable for the request, while deterministic local retrieval over the same corpus remains available when Azure / Chroma is unavailable or skipped. The pipeline then generates a dashboard-facing assignment_context for Streamlit display.

The proposal text supports the RFP Assignment Tool by enabling retrieval of similar historical or sample examples. For a new RFP, the intended system retrieves related proposal chunks and combines retrieval evidence with structured capacity data, service-domain overlap, and risk flags. In Week 4, the RFP page was updated to pass real or pre-built capacity_df into generate_assignment_context() when available, so recommended directors can be capacity-aware rather than purely mock-backed. If only synthetic capacity data is available, the dashboard labels this clearly and treats the output as fallback prototype guidance.

After CGI provided the remaining Azure configuration values, Azure embedding access was validated with a real smoke test, and the Azure embedding plus Chroma retrieval path was integrated into the active RFP engine and Streamlit RFP flow. The RFP engine can now prefer Azure-generated embeddings and Chroma retrieval when Azure configuration is available, while preserving deterministic local fallback retrieval when Azure embedding, Chroma storage, or Chroma querying is unavailable. The Streamlit RFP page displays the retrieval mode so stakeholders can distinguish Azure-backed retrieval from fallback retrieval.

2.4 EDA Findings and Design Implications

The EDA directly influenced several early design decisions. First, because opps2 has the richer CRM schema and preserves the opportunity-level grain more cleanly, it is used as the base table for opportunity_df. Second, because opps1 contains useful service and product fields but also duplicated opportunity IDs, it is treated as a supplemental source rather than a second base table. Third, because the proposal documents are long, the RFP pipeline uses chunking for the historical or

Data source	Approx. size	Main contents	Role in project
opps1 CRM export	8,435 rows 20 columns	Supplemental product, service-solution, IP, service-solution revenue, opportunity-level fallback revenue, and delivery territory fields; contains repeated opportunity_id values.	Secondary supplemental source. Selected fields are collapsed to one row per opportunity_id before merging into opps2.
opps2 CRM export	8,695 rows 33 columns	Richer CRM opportunity schema with owner, status, sales stage, probability, revenue, date, duration, and proposal-related fields.	Primary base table for opportunity_df. It defines the opportunity-level grain for capacity scoring and dashboard inputs.
Proposal / response JSON	19 document pairs	Historical proposal and response text.	Used for RFP text preprocessing, chunking, fallback retrieval, assignment-context generation, and Azure embedding / Chroma sample retrieval smoke testing.

Table 1: Overview of the main project data sources used through Week 4. The files are stored locally and are not committed to GitHub because they contain CGI-provided data.

sample corpus before embedding or retrieval-style processing. Finally, because the data does not contain labeled capacity categories or gold-standard RFP assignments, the system uses transparent formula-based scoring and retrieval-supported recommendation logic instead of supervised learning.

By Week 4, these design implications became more concrete for the final prototype. The capacity dashboard continues to rely on opportunity-level CRM data and formula-based workload estimation, while the RFP Assignment Tool combines Azure / Chroma retrieval evidence with structured capacity signals when capacity_df is available. The team validated Azure embedding access, integrated Azure-generated embeddings with Chroma retrieval in the RFP backend, and preserved the local fallback retrieval path for robustness. The final prototype therefore distinguishes between the preferred Azure-backed retrieval path, fallback retrieval behavior, heuristic scoring, LLM-enriched explanation fields, and remaining business-validation limitations.

3 System Overview

The project has two core deliverables. The first is a Director Capacity Dashboard. This dashboard uses historical CRM opportunity data to estimate each director’s current workload, compare the current workload against the director’s historical baseline, and display capacity labels such as Available, Near Historical Norm, High Load, Overextended, and No baseline. In the Week 4 prototype, relative_load remains the primary

business-facing workload metric because it directly compares a director’s current workload against that same director’s historical norm.

The dashboard also exposes drivers of the score, including open opportunities, late-stage opportunities, weighted pipeline, and inferred delivery commitments. This is important because stakeholders need not only a final capacity label, but also an explanation of why a director appears available, near their normal workload, highly loaded, or overextended. The dashboard keeps capacity_score in the backend for compatibility, but the main dashboard interpretation is now based on relative_load.

The second deliverable is an RFP Assignment Tool. For a new RFP, the tool estimates effort, retrieves similar historical or fallback proposal examples, and recommends directors using retrieval evidence, available capacity signals, service-domain overlap, and risk flags. By Week 4, the RFP page passes real or pre-built capacity_df into generate_assignment_context() when available, so the recommended directors can be capacity-aware rather than purely mock-backed. The RFP output also supports Azure OpenAI chat enrichment for the RFP summary, effort estimate, and director match reasons. If chat enrichment is unavailable, fails, or returns malformed output, the system falls back to deterministic heuristic output with the same dashboard-facing schema.

The current architecture is modular. The opportunity cleaning pipeline produces a local opportunity_df and a cleaned downstream

dataset for validation and scoring. The capacity scoring module consumes this data to compute director-level workload, `relative_load`, `capacity_score`, and capacity labels. The Streamlit dashboard displays capacity metrics and provides an interface for RFP assignment. The RFP pipeline preprocesses proposal text, creates historical or sample chunks for retrieval, supports local fallback retrieval, and produces a dashboard-facing `assignment_context` schema for downstream recommendation display.

After CGI provided the remaining Azure configuration values in Week 4, Azure embedding access was validated through a real smoke test, and Azure-generated embeddings were successfully added to and queried from Chroma. This retrieval path was then integrated into the active RFP engine and Streamlit RFP flow as the preferred retrieval mode when Azure configuration is available. The local fallback retrieval path remains available if Azure or Chroma is unavailable, and the RFP page labels the active retrieval mode. Therefore, the final prototype should be interpreted as a transparent stakeholder-facing decision-support prototype with real capacity-aware recommendations, LLM-enriched output when configured, Azure-backed retrieval when available, fallback-safe behavior, and clearly documented limitations.

4 Weekly Implementation Progress

4.1 Week 1: Data Foundation and Module Skeletons

During Week 1, the team focused on building the first reproducible data foundation and creating skeletons for the main system components. The goal was not to finalize the full capacity model, dashboard, or RFP assignment tool. Instead, we prioritized stable data structures, module boundaries, and interfaces that can support later development.

The work was divided across five roles. Yixiao focused on opportunity data merging, schema comparison, and duplicate / unmatched record summaries. Kian focused on validation of revenue, date, duration, probability, status, and sales-stage fields. Lyken focused on the capacity scoring skeleton and director table prototype. Freya focused on the Streamlit dashboard skeleton and RFP page placeholder. Jai focused on the RFP preprocessing and vector store skeleton.

For the opportunity data merge, we implemented a reproducible pipeline that loads the two

Merge statistic	Value
opps1 rows	8,435
opps2 rows	8,695
opps1 unique IDs	7,679
opps2 unique IDs	8,695
Matched opportunity IDs	7,628
opps1-exclusive IDs	51
opps2-only IDs	1,067
Final opportunity_df rows	8,746
Duplicate IDs in final output	0

Table 2: Week 1 opportunity merge audit summary. The final output preserves opportunity-level grain by avoiding one-to-many expansion from duplicated opps1 records.

anonymized CRM Excel exports, normalizes column names, compares schemas, uses opps2 as the base opportunity-level table, and supplements selected fields from opps1. The pipeline uses `opportunity_id` as the required join key.

A key Week 1 finding was that opps1 contains duplicate `opportunity_id` rows. These duplicates likely reflect service-level or product-level detail rather than separate opportunities. A direct merge from opps1 into opps2 created one-to-many expansion, which would risk double-counting workload in downstream capacity scoring.

To preserve the intended opportunity-level grain, we collapsed opps1 supplemental fields to one row per `opportunity_id` before merging. Duplicate and detail rows from opps1 are preserved in local audit outputs instead of multiplying the final `opportunity_df`. The final local `opportunity_df` contains 8,746 rows, matching the expected no-expansion count of 8,695 opps2 base rows plus 51 opps1-exclusive opportunity IDs. The final merged dataset contains zero duplicate `opportunity_id` rows.

All generated processed data files are kept local and are not committed to GitHub because they are derived from sensitive CGI data. The reproducible code and tests are committed instead, so other team members with local access to the source files can regenerate the same outputs.

4.2 Week 2: From Skeletons to First Integrated Prototype

In Week 2, the team moved from separate data and module skeletons toward a first integrated prototype. The main development path connected the cleaned opportunity data to validation, owner-level capacity scoring, and Streamlit dashboard display. In parallel, the RFP pipeline progressed from place-

holder functions toward sample text preprocessing, chunking, and retrieval-style output.

Yixiao refined the Week 1 merge output into a cleaner downstream dataset for validation, scoring, and dashboard development. This work focused on stabilizing `cleaned_opportunity_df`, confirming the required columns for downstream modules, preserving source and quality flags, and preparing an owner-level base summary using `opportunity_owner`. The goal was not to redesign the Week 1 merge logic, but to make the merged data easier for Kian's validation work, Lyken's scoring module, and Freya's dashboard integration to consume.

Kian extended the validation work from basic field inspection to scoring-oriented field assessment. The validation process reviewed revenue fields, date and duration fields, probability, status, sales stage, `opportunity_owner`, and `opportunity_manager`. The output separates fields that are reliable, fields that are usable with fallback assumptions, and fields that remain risky for formal scoring. This validation layer is important because the capacity model depends on transparent assumptions rather than supervised labels.

Lyken converted the Week 1 `capacity_engine.py` skeleton into a first working capacity scoring workflow. The scoring module computes sales load, inferred delivery load, current owner-level load, historical baseline, relative load, bounded capacity score, and capacity labels. The resulting director-level table provides the first version of `director_capacity_df`, including fields such as `current_load`, `historical_avg_load`, `relative_load`, `capacity_score`, and `capacity_label`.

Freya updated the Streamlit dashboard from a mock-data skeleton toward an integrated capacity dashboard. The Week 2 dashboard reads either the first `director_capacity_df` output or a schema-aligned sample output with the same expected columns. The dashboard displays capacity score by director, capacity labels, current load versus historical baseline, and an owner-level capacity table. Filters and explanation placeholders were also retained so that the interface can continue to evolve as scoring assumptions are refined.

Jai advanced the RFP pipeline from skeleton files toward an independently runnable prototype. The Week 2 work focused on RFP text preprocessing, chunk generation, chunk metadata structure, and a stable `assignment_context` output schema for

dashboard integration. The prototype can produce sample chunks and basic or mock retrieval-style output. Full Azure OpenAI and Chroma integration remain later steps, but the Week 2 interface gives the dashboard a stable structure for retrieved examples, recommended directors, risk flags, and explanatory notes.

Overall, Week 2 established the first end-to-end project workflow: cleaned opportunity data supports validation; validation findings inform capacity scoring; capacity scoring produces a dashboard-ready director table; and the dashboard begins displaying capacity outputs. The RFP pipeline remains partly independent from the capacity workflow, but it now has a clearer preprocessing and output contract for future recommendation integration.

4.3 Week 3: Capacity Dashboard Refinement and RFP Assignment Integration

In Week 3, the team shifted the main development focus from the opportunity data foundation and capacity dashboard prototype toward the RFP Assignment Tool. At the same time, the Director Capacity Dashboard received an important refinement: the dashboard now treats `relative_load` as the primary business-facing workload metric instead of `capacity_score`. This change better communicates overextension because `relative_load` compares a director's current workload against that same director's historical norm, while `capacity_score` is capped once a director reaches or exceeds baseline.

The Week 3 capacity dashboard update also clarified the workload formula and label interpretation. Current load is represented as the sum of sales load and delivery load. Sales load uses open opportunities, stage weight, probability, authoritative revenue, and duration weight. Delivery load uses won active deliveries and monthly revenue. Capacity labels are now assigned from `relative_load`: *Available*, *Near Historical Norm*, *High Load*, *Overextended*, and *No baseline*. These thresholds are project-side prototype choices and should be reviewed by CGI before production use.

The dashboard filtering logic was also refined. Territory and opportunity-owner filters define the selected director population for both current workload and historical baseline. Row-level filters such as status, sales stage, opportunity type, sales model, and created-date range narrow the current workload and trend views but do not recompute the histori-

Module	Owner	Week 2 update	Main output
Data integration	Yixiao	Refined the Week 1 merge output into a downstream-ready cleaned dataset.	<code>cleaned_opportunity_df</code> , column dictionary, quality flags, and owner-level summary.
Data validation	Kian	Validated fields needed for capacity scoring and dashboard use.	Missingness summary, reliable / risky fields, revenue fallback hierarchy, and date / duration assumptions.
Capacity scoring	Lyken	Implemented the first working owner-level capacity scoring workflow.	<code>director_capacity_df</code> , relative load, capacity score, capacity label, and scoring sanity checks.
Dashboard	Freya	Connected the dashboard to real or schema-aligned scoring output.	Capacity score chart, owner-level table, filters, and explanation placeholders.
RFP pipeline	Jai	Built an independently runnable RFP preprocessing and retrieval-style prototype.	Sample chunks, mock or basic retrieval output, and <code>assignment_context</code> schema.
Consistency pass	Yixiao, Freya	Aligned architecture, capacity scoring, dashboard behavior, RFP retrieval, and output contracts after the latest RFP/capacity merge.	Fixed scoring gates, active-quarter baseline/trend logic, dashboard live-data path, stale RFP retrieval, and related docs/tests.

Table 3: Week 2 progress from separate skeletons toward the first integrated prototype.

cal baseline denominator. This keeps the baseline stable and makes filtered views answer a clearer business question: how the filtered current workload compares with the selected directors’ normal workload.

In parallel, the RFP pipeline became the main Week 3 development focus. Based on repository review, the local RFP prototype is partially ready: it can process available proposal text, create chunks, run local fallback embedding or in-memory retrieval, and generate a dashboard-facing `assignment_context`. The active demo path is fallback / heuristic rather than Azure-backed. The remaining integration work focuses on confirming Azure API configuration, completing real Azure OpenAI embedding smoke tests, connecting Chroma retrieval end to end, wiring real capacity data into the RFP engine, refining recommendation logic, and continuing Streamlit RFP page display work.

Yixiao coordinated the Week 3 RFP integration and documentation work. This included reviewing which RFP pipeline components currently work locally, confirming the intended API and fallback retrieval behavior, aligning the `assignment_context` schema with Jai and Freya, and documenting reproducibility, assumptions, blockers, and risks. A key schema decision is that the backend can keep the current field name effort, while the dashboard displays this object as Estimated Effort. If time allows, `estimated_effort` can be added later as an alias

without disrupting existing tests.

Kian completed Sprint 3 implementing a reproducible RFP validation workflow over `proposals_responses.json`. The validation confirmed 19 top-level records and 46 extracted documents (19 proposal documents and 27 response documents), with no malformed top-level records in the current local dataset. The work formalized a chunk metadata contract and validated required fields (`proposal_id`, `chunk_id`, `document_id`, `source_type`, `chunk_index`, `text`, `opportunity_owner`, `opportunity_id`) across 1,216 generated chunks, with zero missing required fields. A key finding is that proposal chunks currently cannot be reliably linked to directors or opportunities because `opportunity_owner` and `opportunity_id` are null in all validated chunks. This limitation is documented explicitly rather than patched with unreliable inferred mapping. Deliverables include `src/rfp_data_validator.py`, `scripts/validate_rfp_data_and_export_samples.py`, `docs/rfp_data_validation.md`, and exported artifacts under `outputs/rfp_validation/` for Lyken/Jai handoff. Export defaults use redacted text and anonymized IDs for safe sharing.

Lyken focused on the Azure embedding and vector-store path. The Week 3 goal was to move beyond local heuristic embedding toward Azure OpenAI embeddings and Chroma retrieval, while keeping local fallback retrieval available. This work included confirming environment-variable requirements, implementing or updating the Azure client,

Track	Owner	Week 3 focus	Main output
Capacity dashboard refinement	Freya	Implemented and documented the relative_load-first dashboard interpretation and updated capacity-label semantics.	Updated dashboard charts, filter semantics, workload formulas, documentation, and tests.
RFP integration / QA	Yixiao	Coordinated RFP pipeline status, API / fallback behavior, assignment schema, smoke testing, and documentation.	API usage notes, finalized assignment_context schema, smoke test checklist, integration QA notes, and risks.
RFP data validation	Kian	Validated proposals_responses.json, text extraction quality, chunk metadata, and proposal-to-director linkage limitations.	JSON structure summary, chunk metadata dictionary, sample chunks, and RFP data validation notes.
Azure embedding / vector store	Lyken	Advanced the code path from local fallback toward Azure embeddings and Chroma while preserving fallback retrieval; real Azure smoke testing remained blocked by missing API version and deployment configuration.	Azure client helper updates, vector store structure, fallback retrieval behavior, and documented blockers for Azure / Chroma completion.
RFP dashboard integration	Freya	Moved the Streamlit RFP page from placeholder toward displaying assignment_context.	RFP input UI, result display, retrieved examples, recommended directors, risk flags, and display notes.
RFP assignment logic	Jai	Improved generate_assignment_context() toward heuristic recommendation output using retrieval examples, available capacity fields when supplied, and risk flags.	Stable assignment output, heuristic effort logic, recommendation notes, risk flag logic, and test updates.

Table 4: Week 3 progress toward RFP Assignment Tool integration.

connecting embeddings to `vector_store.py`, and documenting how the system should behave when Azure is unavailable. At the end of Week 3, real Azure embedding smoke testing was still blocked because the required API version and embedding deployment name had not yet been confirmed, so fallback retrieval remained the stable path for RFP dashboard development.

Freya focused on the RFP page in Streamlit. The Week 3 target was to move the RFP Assignment Tool page beyond a placeholder by displaying `assignment_context` fields such as RFP summary, estimated effort, retrieved examples, recommended directors, capacity signals, match reasons, supporting chunks, and risk flags. The page also needed to handle empty retrieval results or API failures gracefully and clearly label prototype, heuristic, fallback, or mock outputs.

Jai focused on improving `generate_assignment_context()` and the recommendation logic. The Week 3 output used retrieved examples as supporting evidence and could incorporate capacity fields when director data was supplied. However, the Streamlit RFP page had not yet been updated to pass the loaded

capacity data into the engine, so the Week 3 recommendations were still best described as heuristic or mock-backed prototype output rather than fully capacity-aware assignment guidance.

Overall, Week 3 moved the project from a capacity-dashboard-centered prototype toward a broader integrated decision-support tool. The dashboard gained a clearer relative-load-first business interpretation, while the RFP Assignment Tool gained a more explicit path from local fallback retrieval toward Azure embedding, Chroma retrieval, assignment schema stabilization, recommendation logic, and Streamlit display. These Week 3 limitations directly shaped the Week 4 priorities: final integration QA, real capacity-data handoff into the RFP engine, Azure configuration follow-up, LLM enrichment, retrieval smoke testing, and clearer documentation of fallback and prototype behavior.

4.4 Week 4: Final Prototype Stabilization and Architecture-Complete RFP Flow

In Week 4, after CGI provided the remaining Azure configuration values, the team moved from a polished fallback prototype toward an architecture-complete stakeholder-facing prototype. The goal

was still not a production-ready or business-validated assignment system, but rather to complete the intended prototype architecture: real capacity-aware RFP recommendations, Azure OpenAI embedding and Chroma retrieval in the active RFP flow when configured, Azure OpenAI chat enrichment for summary and explanation fields, preserved local fallback behavior, and clear limitations.

Yixiao led final integration QA, scope freeze, and documentation. This work defined the final prototype scope, demo path, claims the team can honestly make, claims the team should not overstate, architecture compliance notes, reproducibility notes, demo checklist, QA checklist, final limitations, and Week 4 report notes. The documentation distinguishes real local processing, heuristic scoring, Azure-backed retrieval, fallback retrieval, mock or sample data, Azure OpenAI chat enrichment, and remaining production-readiness limitations. Yixiao also coordinated final QA evidence from the other team members so the report could accurately reflect the implemented system rather than overclaiming business validation.

Kian led final data validation and retrieval-evidence validation. Kian reviewed the proposal and response data structure, chunk metadata, and proposal-to-director / proposal-to-opportunity linkage limitations. For the architecture-complete Week 4 flow, Kian also validated representative Azure / Chroma retrieval examples, checked whether retrieved chunks were semantically relevant to sample RFP inputs, compared retrieval behavior against the local fallback path where useful, and documented which retrieval results could support summary or similarity claims. A key limitation remains that retrieved proposal chunks should not be overinterpreted as proven director-experience evidence when opportunity_owner or opportunity_id linkage is incomplete.

Lyken led the Azure / Chroma retrieval backend integration. After the required Azure configuration values became available, Lyken confirmed the Azure environment, validated Azure embedding access, and moved the Azure OpenAI embedding plus Chroma retrieval path from smoke-test status into the active RFP backend. The RFP engine now prefers Azure-generated embeddings and Chroma retrieval when the required configuration is available, while preserving deterministic local fallback retrieval if Azure embedding, Chroma storage, or Chroma querying is unavailable. Lyken also doc-

umented retrieval-mode behavior, vector-store rebuild/query steps, backend limitations, and the fallback path that keeps the final demo reproducible.

Freya led final RFP output quality, dashboard polish, and Azure OpenAI chat enrichment. The RFP page was updated to a Week 4 final prototype presentation, including clearer retrieval, capacity-data, and recommendation-status labels; upload/paste workflow labels workflow labeling; a consolidated capacity/retrieval notice; and severity-specific risk flag styling. Freya connected the RFP page to real or pre-built capacity data by passing capacity_df into generate_assignment_context() via director_df=capacity_df. This allows recommended directors to use real capacity signals when available rather than relying only on mock director data. When Azure chat is configured and RFP text is non-empty, generate_assignment_context() can use the chat deployment to enrich the RFP summary, effort estimate, and per-director match reasons. If chat enrichment is unavailable, raises an error, returns malformed JSON, returns partial JSON, or returns invalid nested fields, the system falls back to deterministic heuristic output while preserving the same dashboard-facing assignment_context schema. The output also preserves backward compatibility by keeping effort["rationale"] aligned with effort["reason"].

Jai led RFP page implementation support and stability testing. Jai verified that the Streamlit RFP page can display the agreed assignment_context fields, including RFP summary, Estimated Effort, retrieved examples, similar RFPs, recommended directors, risk flags, and notes. Jai also checked that recommended-director cards can display capacity label, capacity score, relative load, assignment score, match reason, capacity explanation, experience-match explanation, and supporting chunks. For the architecture-complete flow, Jai verified that the page labels the active retrieval mode, remains stable when Azure / Chroma falls back to local retrieval, handles empty input and missing fields, and does not crash when LLM enrichment or capacity data is unavailable.

A key Week 4 integration fix connected the Director Capacity Dashboard output to the RFP Assignment Tool. The Streamlit app was already loading capacity_df, but the RFP page had previously called generate_assignment_context(rfp_text)

without passing the loaded capacity data into the assignment engine. The call was updated to `generate_assignment_context(rfp_text, director_df=capacity_df)`, allowing recommended directors to use real or pre-built capacity signals when available. The RFP results area also tracks and displays the capacity data source used for the recommendation, distinguishing live CRM capacity data, pre-built `director_capacity_df.csv`, and synthetic mock or fallback capacity data.

After the Azure embedding and Chroma retrieval path was integrated, the active RFP flow became architecture-complete at prototype level. The preferred path now uses Azure OpenAI embeddings and Chroma retrieval when configured, then combines retrieved examples with real capacity data and optional Azure chat enrichment to produce the dashboard-facing `assignment_context`. The local fallback retrieval path remains available for reproducibility and failure handling. The Streamlit RFP page labels retrieval status so stakeholders can distinguish Azure-backed retrieval from fallback retrieval.

Local QA and manual demo validation were completed after the Week 4 changes. The Director Capacity Dashboard launched successfully, loaded live CRM capacity data, and displayed director-level capacity outputs including capacity labels, KPI cards, a relative-load chart, and a label-mix chart. The RFP Assignment Tool accepted pasted sample RFP text, retrieved relevant examples through the active retrieval path when configured, generated an RFP summary, displayed estimated effort, recommended directors using live or pre-built capacity data when available, displayed risk flags, and labelled the retrieval and capacity-data status. The final full-suite test result should be reported from the last Week 4 QA run.

Overall, Week 4 moved the project from a Week 3 fallback / heuristic RFP prototype to an architecture-complete stakeholder-facing capstone prototype. The capacity dashboard remained stable around the relative-load-first interpretation. The RFP Assignment Tool became capacity-aware when real or pre-built capacity data is available, integrated Azure OpenAI embedding and Chroma retrieval into the active RFP flow when configured, gained Azure OpenAI chat enrichment for summary, effort, and match-reason fields, preserved local fallback behavior for robustness, and added stronger empty-state, error-state, retrieval-quality,

and response-quality checks.

5 Current Methodology and Prototype Design

The current capacity model is transparent and formula-based. Current workload is represented as the sum of sales load and inferred delivery load. Sales load uses open opportunities, stage weight, probability, authoritative revenue, and project duration. Delivery load uses won active deliveries and monthly revenue as a proxy for delivery commitment. This approach keeps the scoring logic interpretable and avoids using an LLM as the source of truth for numeric capacity scores.

In the Week 4 prototype, `relative_load` remains the primary business-facing capacity metric. It is defined as a director's current workload divided by that same director's historical average workload. This interpretation is useful for communicating overextension because it can distinguish a director who is slightly above normal workload from one who is carrying multiple times their historical norm. `capacity_score` remains in the backend output for compatibility and for downstream RFP assignment scoring, but it is no longer treated as the main dashboard metric because it is capped once `relative_load` reaches or exceeds the baseline.

Capacity labels are assigned from `relative_load`. The current prototype labels directors as *Available*, *Near Historical Norm*, *High Load*, *Overextended*, or *No baseline*. These thresholds are project-side calibration choices based on the observed director-level distribution and the business interpretation that carrying at least twice the historical workload should be considered overextended. CGI should review and adjust these thresholds before production use.

The dashboard filtering logic is designed to keep the historical baseline stable. Territory and opportunity-owner filters define the selected director population for both current workload and historical baseline. Row-level filters such as status, sales stage, opportunity type, sales model, and created-date range narrow the current workload and trend views, but do not recompute the historical baseline denominator. This allows filtered views to answer how a selected slice of current workload compares with the selected directors' normal workload.

The Streamlit dashboard delegates capacity calculations to shared functions in `capacity_engine.py`. For live filtered views, the

Track	Owner	Week 4 focus	Main output
RFP contract consistency pass	Yixiao, Freya	Tightened the Week 3 RFP assignment contract before final integration.	Stable assignment_context; effort.reason primary and rationale alias; safer fallback chunks; non-finite numeric cleanup; regression tests; 108 passed.
Final integration QA	Yixiao	Froze final prototype scope, demo path, safe claims, limitations, and reproducibility notes.	QA checklist, demo checklist, architecture-compliance notes, final caveats, and report-ready scope language.
RFP data validation	Kian	Validated proposal structure, chunk metadata, retrieval examples, and linkage limits.	Final validation summary, representative retrieval examples, chunk metadata notes, and proposal-to-director / opportunity linkage caveats.
Azure / Chroma retrieval	Lyken	Integrated Azure OpenAI embeddings and Chroma retrieval into the active RFP flow, with fallback preserved.	Azure environment check, embedding validation, Chroma retrieval path, retrieval-mode notes, and fallback behavior.
RFP output quality and LLM enrichment	Freya	Polished RFP UI, wired real capacity data, and added Azure chat enrichment.	Week 4 labels, upload/paste labels, severity risk flags, capacity_df handoff, LLM-enriched summary/effort/match reasons, and fallback-safe tests/docs.
RFP page stability	Jai	Verified page display and stability across normal, fallback, missing-field, empty, and error states.	Assignment-context display checks, sample RFP tests, retrieval-mode display, and stability notes.
Final prototype validation	Freya	Finalizes the architecture-complete prototype.	Improved upload, retrieval, service-fit, LLM enrichment, and fallback behavior; documented in final_prototype_validation.md.

Table 5: Week 4 progress toward final prototype stabilization and architecture-complete RFP assignment flow.

interface rebuilds current workload from filtered opportunity rows while keeping the historical baseline anchored to the selected owner / territory population. This keeps the scoring logic centralized in the engine while allowing the dashboard to support interactive filtering. In Week 4, the resulting capacity_df is also passed into the RFP assignment engine when available, connecting the Director Capacity Dashboard output to capacity-aware director recommendations.

For RFP assignment, the current prototype combines proposal text preprocessing, retrieval-oriented evidence, structured capacity signals, service_solution fit, heuristic scoring, and optional LLM enrichment. The RFP pipeline extracts proposal or response text from proposals_responses.json when available, creates chunks for the historical corpus, and uses a built-in sample corpus only when the local proposal corpus is unavailable. New RFP input is accepted through pasted text or TXT, DOCX, and text-based PDF upload. The submitted RFP text

is chunked before retrieval, and the dashboard-facing assignment_context is produced from the retrieved examples, capacity data, service-fit signal, risk flags, and optional LLM enrichment.

The intended RFP recommendation logic combines retrieval evidence with structured director capacity signals. In Week 4, the Streamlit RFP page was updated to call generate_assignment_context(rfp_text, director_df=capacity_df) when capacity data is available. This allows recommended directors to use real or pre-built capacity signals, including capacity_label, capacity_score, and relative_load, rather than relying only on mock director data. If only synthetic mock capacity data is available, the dashboard labels this clearly and the recommendation should be interpreted as fallback prototype guidance.

The RFP output uses a stable assignment_context schema for dashboard display. This schema includes an RFP summary, an effort object displayed as Estimated Effort,

retrieved examples, recommended directors, risk flags, and notes. Recommended-director cards can display capacity label, capacity score, relative load, assignment score, match reason, capacity explanation, experience match explanation, and supporting chunks. This design keeps the RFP page stable even when individual backend components use real data, fallback data, heuristic logic, or optional LLM enrichment.

Week 4 also added Azure OpenAI chat enrichment. When Azure chat configuration is available, `generate_assignment_context()` can use the chat deployment to enrich the RFP summary, effort estimate, and per-director match reasons. When chat enrichment is unavailable, fails, or returns malformed JSON, the system falls back to deterministic heuristic output using the same schema. This keeps the dashboard output robust while improving readability when LLM enrichment succeeds. LLM-generated fields are still treated as prototype-quality explanations rather than validated business analysis.

The preferred retrieval architecture uses Azure OpenAI embeddings with Chroma vector-store retrieval. After CGI provided the remaining Azure configuration values, Azure embedding access passed a real smoke test, sample Azure + Chroma retrieval was validated, and the Azure / Chroma retrieval adapter was integrated into the active RFP engine and Streamlit RFP flow. When Azure configuration is available, the RFP engine uses Azure-generated embeddings and Chroma retrieval as the preferred path. When Azure or Chroma is unavailable, the system falls back to deterministic local retrieval while preserving the same dashboard-facing `assignment_context` schema.

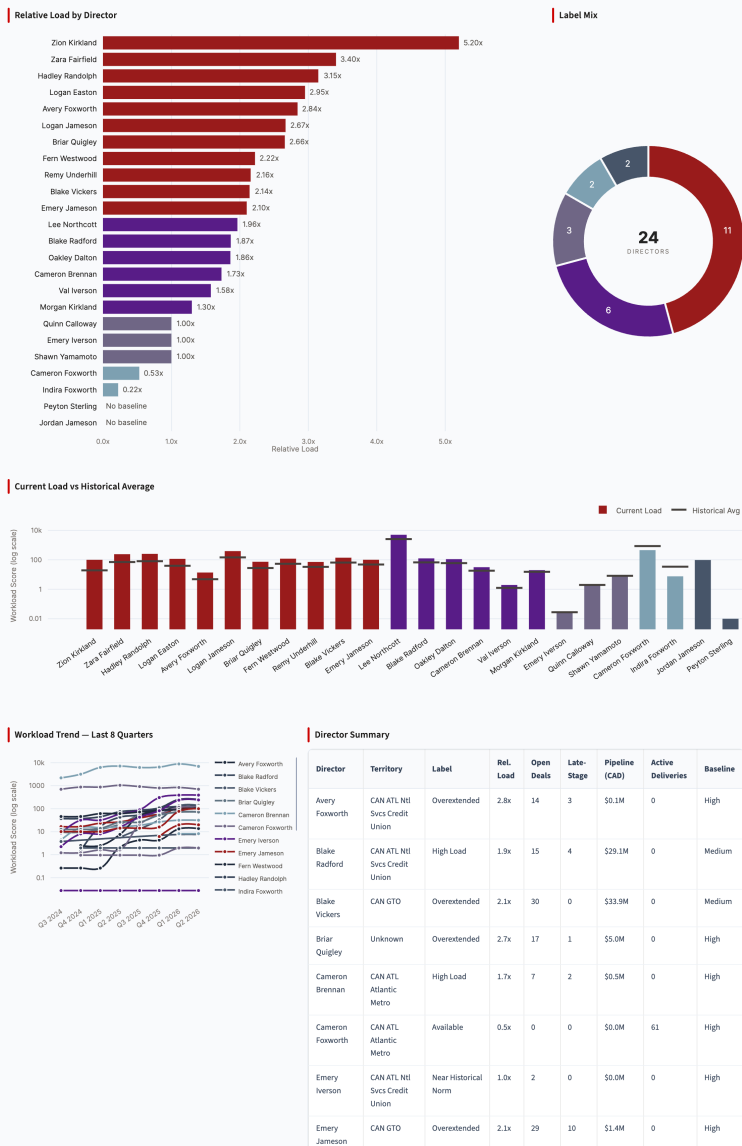
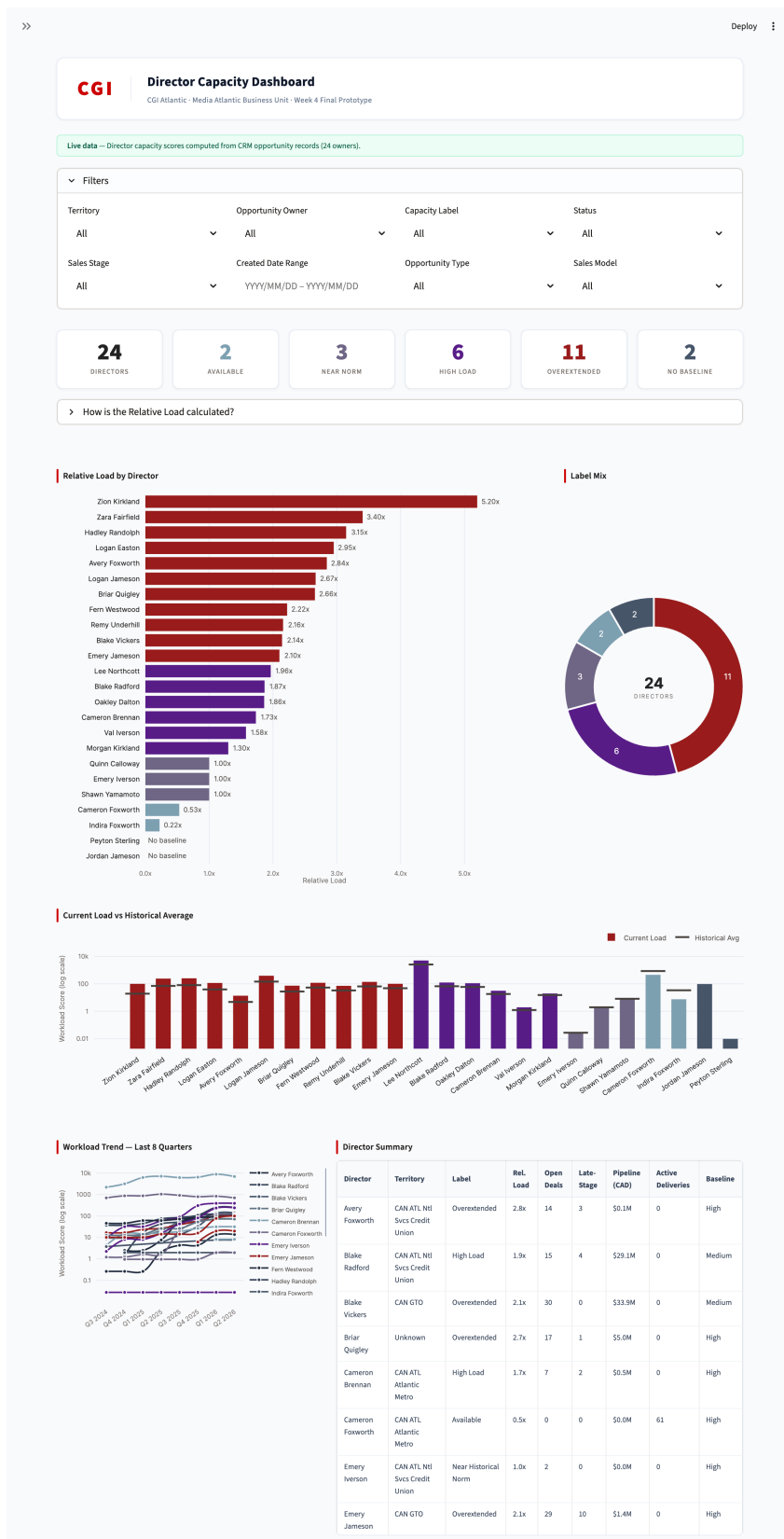


Figure 1: Week 4 Director Capacity Dashboard prototype.

>>

Deploy

CGI

RFP Assignment Tool

CGI Atlantic - Media Atlantic Business Unit - Week 4 Final Prototype

Live data — Director capacity scores computed from CRM opportunity records (24 owners).

Week 4 Final Prototype — RFP Assignment Tool

Retrieval: active mode is shown after analysis (Azure/Chroma when available, local fallback otherwise). Capacity data: real director capacity scores when available. Recommendations: prototype guidance only — not final CGI assignment decisions.

RFP Input

Upload RFP document

20-09aa...-final.pdf

Loaded text from 20-09aa-rfp-for-case-management-software-final.pdf.

Upload a supported document or paste RFP text directly below.

REQUEST FOR PROPOSALS FOR
CITY ATTORNEY'S OFFICE
CASE MANAGEMENT SOFTWARE SOLUTION

RFP NUMBER ISSUED SEPTEMBER 8, 2020

PROPOSALS DUE OCTOBER 1, 2020

Territory

Any

Service Domain

Any

Territory and service domain are UI context only for now; backend scoring does not use them yet.

Analyze RFP

Retrieval mode — Local fallback retrieval active.

Prototype output — capacity-aware recommendations using Live CRM capacity data. Retrieval: Local fallback retrieval active.

Azure/Chroma rebuild skipped for this full-corpus dashboard request; using local retrieval over the same corpus.

Retrieval corpus: local proposals_responses.json corpus; 1216 corpus chunks; 11 pasted-RFP query chunks.

Retrieved chunks do not include reliable opportunity_owner/opportunity_id linkage, so they are semantic evidence rather than proof of director experience.

RFP Summary

The City of Duluth is seeking proposals for a case management software solution for its City Attorney's Office, including software, hardware, implementation, training, and ongoing support to replace the current system.

Estimated Effort

High

Estimated duration: 8-12 weeks

The project involves multiple components including software acquisition, hardware setup, implementation, training, and long-term support.

Similar Historical RFPs

1. 9_sr_25_fy22_professional_information_technology_services — similarity 1.00

2. 9_sr_25_fy22_professional_information_technology_services — similarity 1.00

3. 9_sr_25_fy22_professional_information_technology_services — similarity 1.00

Recommended Directors

1. Indra Foxworth

Capacity: Available

Capacity score: 0.777846221831099

Relative load: 0.22215377816890106

Assignment score: 0.81

Indra has extensive experience in implementing secure and user-friendly software solutions, making her a strong fit for this project's focus on confidentiality and ease of access.

Capacity explanation

Indra Foxworth is labelled Available with capacity score 0.78 and relative load 0.22. This is a heuristic input to the assignment score, not a final staffing decision.

Experience match explanation

Experience fit is estimated from retrieved historical/sample RFP chunks and simple service-keyword overlap (0.40).

Supporting chunks:

9_sr_25_fy22_professional_information_technology_services_response_proposal_response_1_chunk_0013

9_sr_25_fy22_professional_information_technology_services_response_proposal_response_1_chunk_0011

2. Cameron Foxworth

3. Jordan Jameson

Risk Flags

Low: Retrieved proposal chunks do not include reliable opportunity_owner or opportunity_id linkage. Treat them as semantic similarity evidence, not proof of a director's prior experience.

Medium: Some directors have no historical baseline, so capacity_score and relative_load use heuristic defaults in RFP ranking. Review No baseline candidates before assignment.

High: At least one candidate director is overextended based on capacity signals. This should be reviewed before assignment.

Notes

Prototype output enriched with Azure OpenAI chat analysis. Retrieval used local fallback. Retrieved evidence searched the local proposals_responses.json corpus.

Figure 2: Week 4 RFP Assignment Tool prototype.

6 Project Management and Next Steps

In Week 4, the team's project management focus shifted from feature expansion to final prototype stabilization, scope closure, demo readiness, integration QA, and final report preparation. The team treated the Director Capacity Dashboard as the stable first deliverable and focused remaining integration effort on making the RFP Assignment Tool architecture-complete, clearer, more reliable, and more honest about what is real, Azure-backed, LLM-enriched, heuristic, fallback-based, mock-backed, or still prototype-level.

Yixiao led final integration QA and scope-freeze work. The team documented the final prototype scope, the official demo path, the claims that can be made safely, the claims that should not be overstated, architecture-compliance notes, reproducibility notes, demo checklist, QA checklist, and final limitations. This documentation was important because the project now includes several execution modes: real local processing for capacity scoring, heuristic scoring logic, Azure / Chroma retrieval when configured, local fallback retrieval, optional sample data, and Azure OpenAI chat enrichment.

Kian led final validation of the RFP evidence base. This included reviewing proposal / response structure, chunk metadata, retrieval examples, and linkage limitations. Kian's validation notes help define what the retrieved examples can support: they can support semantic similarity and retrieval-evidence discussion, but they should not be treated as definitive director-experience evidence when reliable proposal-to-director or proposal-to-opportunity linkage is unavailable.

Lyken led the Azure / Chroma retrieval backend completion. After CGI provided the remaining Azure configuration values, Lyken validated the Azure environment, confirmed Azure embedding access, integrated Azure-generated embeddings and Chroma retrieval into the active RFP backend, and preserved local fallback retrieval for failure handling. This moved Azure / Chroma from sample smoke-test validation into the active RFP prototype flow, while keeping the system reproducible when Azure or Chroma is unavailable.

Freya led stakeholder-facing RFP output quality, dashboard polish, and Azure OpenAI chat enrichment. Freya connected the RFP Assignment Tool to real or pre-built capacity data by passing `capacity_df` into `generate_assignment_context()`, updated the

RFP page labels and risk-flag display, added Azure chat enrichment for RFP summaries, effort estimates, and director match reasons, and preserved safe heuristic fallback behavior when LLM enrichment is unavailable or invalid.

Jai led RFP page implementation support and stability testing. Jai verified that the Streamlit page displays the agreed `assignment_context` fields, labels the active retrieval mode, handles sample RFP inputs, supports empty and error states, and remains stable when Azure / Chroma, LLM enrichment, or capacity data is unavailable. Jai's testing helped confirm that the architecture-complete backend flow can be shown through the dashboard without breaking the stakeholder-facing UI.

The next stage of the project should focus on final evaluation, stakeholder review, demo polish, and handoff rather than major new feature expansion. The capacity dashboard thresholds and label names should be reviewed with CGI to confirm whether they match business language. The RFP Assignment Tool should be evaluated using representative examples, with particular attention to whether retrieved examples support the displayed match reasons and whether risk flags are useful to stakeholders. Future work should production-harden the Azure + Chroma retrieval path, expand retrieval validation across more representative RFPs, connect more complete historical proposal metadata where available, support uploaded RFP documents rather than paste-only input, and calibrate recommendation weights with CGI feedback.

7 Current Assumptions and Risks

Several assumptions and risks remain at the end of Week 4. First, `opportunity_id` is still treated as the required join key for opportunity-level merging. Second, `opps2` remains the primary base CRM opportunity table, while `opps1` is used only for selected supplemental fields after being collapsed to one row per `opportunity_id`. Third, `opportunity_owner` is treated as the main director-level entity for MVP capacity scoring, while `opportunity_manager` is retained as optional metadata.

The capacity dashboard still depends on inferred workload rather than measured hours, calendar commitments, or manually confirmed director capacity. Current load is estimated from sales pipeline and inferred delivery signals, so the resulting workload estimates should be treated as

directional decision-support indicators rather than exact measurements of utilization or availability.

The Week 4 dashboard continues to use `relative_load` as the primary business-facing workload metric. This improves interpretability, but the current label thresholds remain temporary project-side calibration choices. Labels such as Available, Near Historical Norm, High Load, Overextended, and No baseline should be reviewed by CGI before production use. In particular, the threshold for Overextended reflects the current team interpretation that at least twice the historical workload should be treated as overextension.

The stable-baseline filtering decision also remains an assumption. Territory and opportunity-owner filters define the selected director population for both current workload and baseline, while row-level filters narrow current workload without recomputing the historical baseline. This makes the dashboard easier to interpret, but CGI may later decide that some additional filters should also redefine the baseline population.

The RFP Assignment Tool became capacity-aware in Week 4 when the Streamlit page began passing `capacity_df` into `generate_assignment_context()`. However, the resulting recommendations are still prototype guidance rather than final CGI staffing decisions. The assignment score combines available capacity signals, retrieval evidence, keyword overlap, and heuristic logic. These weights have not yet been calibrated against stakeholder-approved outcomes or historical assignment labels.

The project still lacks ground-truth labels for director capacity, RFP effort, and correct director assignment. Evaluation therefore relies on stakeholder review, threshold sensitivity checks, sanity checks, smoke tests, manual demo validation, and selected case studies rather than standard supervised-learning metrics.

The RFP retrieval evidence also has limitations. Proposal chunks cannot currently be reliably linked to directors or opportunities when `opportunity_owner` and `opportunity_id` are missing from validated chunk metadata. As a result, retrieved examples can support semantic similarity claims, but they should not be overinterpreted as proven director-experience evidence unless reliable linkage is available.

Azure embedding access and Azure + Chroma retrieval were validated and integrated into the active RFP flow in Week 4 after CGI provided the

remaining Azure configuration values. However, this integration remains prototype-level rather than production-hardened. The system preserves local fallback retrieval when Azure embedding, Chroma storage, or Chroma querying is unavailable. Retrieval results should still be evaluated by stakeholders because semantic similarity does not automatically prove director experience, business fit, or final assignment suitability.

Azure OpenAI chat enrichment introduces an additional interpretation risk. When configured, the LLM can enrich the RFP summary, effort estimate, and director match reasons. These fields improve readability, but they are not validated business analysis. If chat enrichment fails or returns malformed output, the system falls back to deterministic heuristic output. The dashboard and report should therefore label LLM-enriched text as prototype explanation rather than authoritative staffing guidance.

The RFP page still has interface limitations. The current workflow accepts pasted RFP text; PDF or DOCX upload parsing is not yet connected. The architecture-complete Week 4 path supports Azure / Chroma retrieval from pasted or sample RFP text, but document ingestion, file parsing, and production-grade RFP upload handling remain future work.

Finally, the system may combine real data, heuristic logic, fallback retrieval, sample data, mock capacity data, Azure smoke-test components, and LLM-enriched text during development and demonstration. These components must remain clearly labeled in the dashboard and documentation so that stakeholders understand which parts are implemented, which parts are fallback or heuristic, which parts are sample-level validation, and which parts still require future integration or business review.

8 Conclusion

This Week 4 version extends the report from a Week 3 RFP integration update into a final stakeholder-facing capstone prototype report. Week 1 established the reproducible opportunity merge, preserved opportunity-level grain, and created the main module skeletons. Week 2 refined the cleaned opportunity data, added validation-oriented field assessment, produced an initial director-level capacity scoring output, connected the dashboard to capacity results, and advanced the RFP pipeline

toward preprocessing, chunking, and retrieval-style output. Week 3 refined the Director Capacity Dashboard around a `relative_load`-first interpretation and shifted the main development focus toward RFP assignment, including fallback retrieval, assignment schema stabilization, heuristic recommendation logic, and Streamlit RFP page integration.

In Week 4, the team stabilized the prototype for final demonstration and reporting. The capacity dashboard remained centered on transparent formula-based workload estimation and relative-load interpretation. The RFP Assignment Tool was updated to pass real or pre-built `capacity_df` into `generate_assignment_context()` when available, allowing the recommendations to use real capacity signals instead of relying only on mock director data. The RFP page also improved stakeholder-facing clarity through explicit fallback, heuristic, prototype, and capacity-data labels; better empty-state and error-state handling; and clearer display of summaries, estimated effort, retrieved examples, recommended directors, risk flags, and notes.

The Week 4 prototype also added Azure OpenAI chat enrichment and integrated the preferred Azure retrieval architecture into the active RFP flow. When Azure chat is configured, the system can enrich the RFP summary, effort estimate, and director match reasons while falling back to deterministic heuristic output if LLM enrichment is unavailable or malformed. After CGI provided the remaining Azure configuration values, Azure embedding access passed a real smoke test, Azure-generated embeddings were successfully added to and queried from Chroma, and the Azure / Chroma retrieval path was integrated into the RFP engine and Streamlit RFP page as the preferred retrieval mode when available. Local fallback retrieval remains available for robustness and reproducibility.

Overall, the project now demonstrates a transparent decision-support workflow: a stakeholder can view director capacity results, inspect workload drivers, paste an RFP, receive capacity-aware prototype recommendations when capacity data is available, review similar retrieved examples, and see risk flags and caveats. The system remains an applied prototype rather than a production-ready or business-validated assignment system. Future work should focus on CGI review of capacity thresholds and label language, calibration of recommendation weights, larger-scale validation of retrieved exam-

ples, reliable proposal-to-director linkage, production hardening of the Azure + Chroma retrieval path, document upload support, and final handoff documentation.